LLM-Assisted Mobile Robot Task Planning with a Validated Skill Interface and Execution Feedback

CandidateInterview Assignment

Abstract

This report presents a single mobile-robot system that converts natural-language instructions into a small, validated sequence of high-level robot skills. The system runs ROS 2 Humble and Webots R2025a inside WSL2 on a Windows host. A task-level planner selects only from a fixed allowlist of three skills; a multi-layer validator checks every proposed call before execution; and a bounded recovery module observes each structured result and decides between retry, obstacle recovery, one-shot LLM replanning with execution feedback, and safe stop-and-report. Experiments show successful pose reaching with a position error of 0.049 m, multi-waypoint execution, safe rejection of invalid and unknown-skill plans, and a closed-loop replanning trace in which the LLM receives the observed timeout and produces a new validated plan.

1 Introduction

Large language models (LLMs) can translate natural language into structured tool calls, but directly executing their output on a robot is unsafe. The assignment requires that an LLM may *select* existing high-level skills but must never issue velocity commands or execute generated code. The design goals are reliability, explicit skill contracts, strict validation, and reproducible experiments.

The proposed system follows the loop `Plan` → `Validate` → `Execute` → `Observe` → `Recover`. The main contributions are:

- a validated skill interface with a fixed allowlist and JSON Schema contracts;
- a ROS 2 implementation of `navigate_to`, `follow_waypoints`, and `stop`;
- a closed-loop task agent (`TaskAgent`) that centralizes planning, validation, execution, observation, and recovery;
- a mock planner and an optional OpenAI-compatible LLM planner that can consume structured execution feedback for bounded replanning;
- bounded recovery for invalid locations, unknown skills, timeouts, and blocked obstacles;
- a reproducible WSL2/Webots setup with JSONL execution logs.

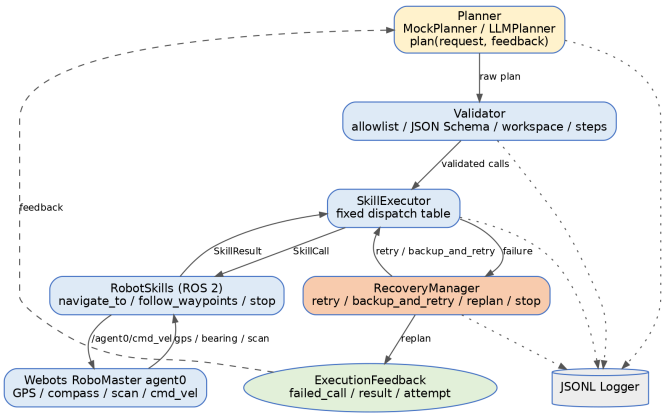


Figure 1: Closed-loop system architecture. The dashed edge is the execution feedback channel used by the LLM planner for bounded replanning.

2 System Architecture

Figure 1 shows the architecture. The planner produces a raw plan (`SkillCall[]`). The validator checks the plan before any robot action. Only validated calls reach the skill executor, which invokes the `RobotSkills` ROS 2 node. The node publishes velocity commands and reads position/heading/scan feedback from Webots. Every skill returns a structured `SkillResult`; on failure, the recovery module decides whether to retry, back up and retry, replan with `ExecutionFeedback`, or stop and report. All events are appended to a JSONL log.

2.1 Modules

The repository separates the task-level agent from the ROS 2 skill layer:

- `agent/task_agent.py`: owns the Plan–Validate–Execute– Observe–Recover loop;
- `agent/planner.py`: mock planner and optional LLM planner;
- `agent/registry.py`: machine-readable skill registry;
- `agent/validator.py`: six-layer validation;
- `agent/executor.py`: fixed dispatch to the three skills;
- `agent/recovery.py`: bounded recovery decisions;

Table 1: Skill interface summary.

Skill	Contract
`navigate_to`	params: x, y, θ; success: position ≤ 0.05 m and heading ≤ 0.10 rad; timeout: 30 s
`follow_waypoints`	params: a list of waypoints; on failure returns the last reached index; timeout: 30 s per waypoint
`stop`	publishes zero velocity; success: zero command published; timeout: 2 s

- `agent/models.py`: `SkillCall`, `SkillResult`, `ValidationResult`, and `ExecutionFeedback`;
- `agent/logger.py`: JSONL event logging;
- `robot_skills/skills.py`: ROS 2 implementation of the skills.

2.2 Skill interface

Table 1 summarises the three skills. The registry is the single source of truth: it generates both the JSON Schema used by the validator and the tool list exposed to the LLM.

3 Implementation

3.1 ROS 2 skill layer

The `RobotSkills` node subscribes to `/agent0/gps` (`PointStamped`) for position and to `/agent0/compass/bearing` (`FloatStamped`) for heading. It publishes `Twist` messages on `/agent0/cmd_vel`. The Webots compass bearing is clockwise-positive, while ROS yaw is counter-clockwise-positive, so the node computes

$$\psi = \text{normalize}(-\text{radians}(\text{bearing})). \tag{1}$$

3.2 Holonomic `navigate_to` with PID

The RoboMaster base is holonomic. Rather than rotating first and then driving, the controller translates directly in the body frame and only rotates in place for the final heading:

$$e_x = x_d - x, \qquad e_y = y_d - y, \tag{2}$$

$$v_x^w = \text{clamp}\big(k_p e_x + k_i \textstyle\int e_x dt + k_d \frac{de_x}{dt}, -v_{\max}, v_{\max}\big), \tag{3}$$

$$v_y^w = \text{clamp}\big(k_p e_y + k_i \textstyle\int e_y dt + k_d \frac{de_y}{dt}, -v_{\max}, v_{\max}\big), \tag{4}$$

$$v_x^b = \cos\psi\, v_x^w + \sin\psi\, v_y^w, \tag{5}$$

$$v_y^b = -\sin\psi\, v_x^w + \cos\psi\, v_y^w. \tag{6}$$

When the position error is above tolerance, the node commands $(v_x^b, v_y^b, 0)$; otherwise it commands $(0, 0, \omega)$ with $\omega = \text{clamp}\big(k_p^\theta e_\theta + k_i^\theta \int e_\theta dt + k_d^\theta \frac{de_\theta}{dt}, -\omega_{\max}, \omega_{\max}\big)$. Success requires

$$\sqrt{e_x^2 + e_y^2} \leq 0.05\,\text{m} \quad \text{and} \quad |e_\theta| \leq 0.10\,\text{rad}. \tag{7}$$

On timeout the node stops the robot and returns `success=false, reason="timeout"`.

3.3 `follow_waypoints` and `stop`

`follow_waypoints` calls `navigate_to` sequentially and tracks `last_reached_index`. If a waypoint fails, it stops and returns that index. `stop` publishes zero velocity and returns a structured result.

3.4 Obstacle safety layer

`navigate_to` also subscribes to `/scan`. Before publishing a translation command, it checks the minimum valid range inside a 30-degree cone around the intended body-frame motion direction. If that distance is at most 0.35 m, the skill stops and returns `success=false, reason="obstacle_blocked"`. The task-level recovery then backs up, rotates, and retries once.

3.5 Closed-loop task agent

`TaskAgent` owns the execution loop. It first asks the planner for a plan and logs the raw plan. The validator checks the plan and rejects it before any motion if it is invalid. For each validated call, the executor invokes the corresponding skill and logs the observed `SkillResult`. Successful calls are recorded and execution continues. For a failure, the `RecoveryManager` chooses a bounded action:

- `retry`: re-run the same call once after a timeout;
- `backup_and_retry`: back up, rotate, and re-run the call once after an obstacle-blocked result;
- `replan`: send `ExecutionFeedback` to the LLM planner and request a new plan (only when the planner supports feedback and the replan budget remains);
- `stop_and_report`: stop the robot and return the failure.

A replanned plan is validated from scratch before any robot motion. If the planner returns the same plan, the agent stops safely instead of looping. The number of replans is bounded by `--max-replans` (default 1).

3.6 Validation layers

The validator applies six layers: plan length, fixed allowlist, JSON Schema, workspace bounds, preconditions, and retry/replan limits. The allowable workspace is $x, y \in [-1.5, 1.5]$ m. Table 2 lists the layers and the corresponding failure reasons.

Table 2: Validator layers and produced reasons.

Layer	Failure reason
Plan length	`too_many_steps`
Allowlist	`unknown_skill`
JSON Schema	`schema_invalid`
Workspace bounds	`target_out_of_workspace`
Preconditions	`precondition_failed`
Retry/replan limit	`stop_and_report`

4 Experiments and Results

4.1 Platform

Experiments were run on Windows 11 with WSL2 Ubuntu 22.04, ROS 2 Humble, and Webots R2025a. The GPU is an AMD Radeon RX 9070 XT (16 GB). WSLg exposes a hardware-accelerated OpenGL 4.2 context backed by the Mesa D3D12 driver: `glxinfo` reports renderer `D3D12 (AMD Radeon RX 9070 XT)` and `Accelerated: yes`; `glxgears` reaches approximately 140 FPS.

4.2 Task-level regression

Table 3 reports the full regression after the closed-loop refactor. The success case reaches storage with a position error of 0.0494 m. The multi-waypoint case reaches both waypoints and reports `last_reached_index=1`. The invalid location is rejected at the workspace layer for the mock planner; for the real LLM, the model returns an empty tool list, which the validator rejects before execution. The unknown skill is rejected at the allowlist layer. The forced timeout is retried once and then safely stopped and reported. The obstacle-recovery case succeeds after one back-up-and-rotate retry, while the persistent-obstacle case stops and reports the target as blocked. The optional cloud LLM planner was tested with an OpenAI-compatible endpoint and model gpt-5.5; it produced correct `navigate_to` and `follow_waypoints` plus `stop` plans.

The key new result is R12. The LLM first plans to the charging station. After two forced timeout observations, the recovery layer sends the structured `ExecutionFeedback` back to the LLM. The LLM returns a new validated plan that navigates to storage instead; the new plan is executed and, because the timeout is still forced to one second, it is safely stopped and reported. The JSONL trace therefore contains the full sequence `raw_plan` → `skill_result` → `recovery_decision` → `feedback_to_planner` → `replanned_plan` → `validation` → `skill_result` → `final_outcome`.

4.3 Manual control and calibration

Table 4 shows the observed response of the reference `wheels_controller`. The controller uses an incorrect wheel-radius constant (0.26 instead of the URDF value 0.06), so the skill layer uses closed-loop feedback rather than open-loop timing.

5 Discussion

5.1 Safety

The LLM never receives direct actuation authority. It can only select from three allowlisted skills, and every proposed call passes through the validator before execution. The executor maps skill names to Python methods with predefined signatures; no generated Python, shell, or ROS command is executed. Targets outside the workspace, unknown skills, malformed arguments, plans that exceed the step limit, and empty plans are rejected. A timeout or blocked obstacle always ends with a zero velocity command and a structured failure report.

5.2 Execution feedback

The refactored agent makes execution feedback explicit and bounded. The LLM sees only task-level information: the original request, the failed call, the structured result, the attempted recovery message, and already completed work. It never sees raw sensor data. The feedback path is rule-triggered, so the system remains deterministic and testable with a mock planner while still supporting a real LLM comparison.

5.3 Reproducibility

The system is reproducible from the repository and the environment notes in `docs/environment.md`. The setup requires ROS 2 Humble, Webots R2025a, the `webots_ros2` 2025.0.0 packages, and the reference `webots_ros2_robomaster` world assets. The Webots asset cache and the reference `protos/meshes` directory are documented because they are large external assets. The generated JSONL logs are committed as examples.

5.4 Limitations

The final configuration runs Webots inside WSL2 through WSLg. The WSLg path has some D3D12 translation and Xwayland compositing overhead, so it is less responsive than the optional native Windows Webots mode, but it is sufficient for the assignment. The reference setup has no `/agent0/odom` topic; the skill layer therefore uses `/agent0/gps` for position and `/agent0/compass/bearing` for heading. The reference `wheels_controller` uses an incorrect wheel radius, so closed-loop control is required. The current LLM replanning is one-shot and rule-triggered; it does not yet compare the original plan with the replanned plan beyond detecting identical plans.

Table 3: Task-level regression results with the closed-loop agent.

ID	Instruction	Planner / mode	Outcome	Skill calls	Recovery
R1	Go to storage.	mock / direct	success, 18.58 s, 0.0494 m	1	none
R2	Visit storage, then workbench, and stop.	mock / direct	success, 38.27 s	2	none
R3	Go to storage.	mock / map (A*)	success, 17.73 s, `map_reached`	1	none
R4	Go to space_station.	mock	rejected	0	abort, `target_out_of`
R5	Go to entrance and fly.	mock	rejected	0	abort, `unknown_skill`
R6	Go to charging_station, forced 1 s timeout	mock / direct	failed	2	retry once, stop and
R7	Go to storage, obstacle then removed	mock / direct	success, 0.0500 m	2	back up, rotate, retry
R8	Go to storage, obstacle remains	mock / direct	failed	2	back up, rotate, stop
R9	Go to storage.	real LLM / direct	success, 16.43 s, 0.0499 m	1	none
R10	Visit storage, then workbench, and stop.	real LLM / direct	success, 38.70 s	2	none
R11	Go to space_station.	real LLM	rejected	0	LLM empty plan, ab
R12	Go to charging_station, forced 1 s timeout	real LLM / direct	failed	4	retry, **replan with f**

Table 4: Observed command response.

Command	Observed motion	Approx. response
`linear.x = 1.0`	0.20 m in 2 s	0.10 m/s
`linear.y = 1.0`	0.22 m in 2 s	0.11 m/s
`angular.z = 5.0`	127° in 2 s	1.11 rad/s

5.5 Next steps

Future work includes an LLM planner that reasons over the full execution trace and explicit failure semantics, dynamic obstacle avoidance using `/scan`, a dedicated odometry topic, and a Docker/WSL one-click setup.

6 Conclusion

This report described a reliable, validated LLM-assisted task-planning system for a simulated mobile robot. The system separates planning, validation, execution, observation, and recovery; enforces a strict allowlist; returns structured results for every skill; and closes the loop by sending bounded execution feedback back to the LLM planner. Experiments demonstrated successful pose reaching, multi-waypoint execution, map-based A* navigation, rejection of invalid locations and unknown skills, obstacle recovery, and a real LLM replanning trace after observed timeouts.

References

[1] ROS 2 Humble documentation. https://docs.ros.org/en/humble/

[2] Cyberbotics Webots. https://cyberbotics.com/

[3] `webots_ros2`. https://github.com/cyberbotics/webots_ros2

[4] `webots_ros2_robomaster`. https://github.com/MohismLab/webots_ros2_robomaster

[5] Microsoft WSLg. https://github.com/microsoft/wslg

[6] Mesa D3D12 driver. https://docs.mesa3d.org/drivers/d3d12.html

[7] OpenAI-compatible function calling. https://platform.openai.com/docs/guides/function-calling